

BÀI THỰC HÀNH ADAPTER PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Tình huống thực tế: Tích hợp hệ thống thanh toán bên thứ ba
 - ✓ Áp dụng Adapter Pattern chuẩn GoF
 - ✓ Xử lý sự khác biệt interface giữa hệ thống nội bộ và external API
-

BÀI TOÁN

Hệ thống E-commerce có sẵn interface thanh toán nội bộ:

```
IPayment
```

Nhưng cần tích hợp thêm:

-  Stripe (API khác)
-  Momo (API khác)

 Vấn đề:

- Mỗi bên thứ 3 có interface KHÁC nhau
- Không thể sửa code hệ thống cũ

 Giải pháp:

 Dùng **Adapter Pattern** để "chuyển đổi interface"

KIẾN TRÚC HỆ THỐNG

```
Frontend
  ↓
OrderService
  ↓
IPayment (Target)
  ↓
Adapter (StripeAdapter | MomoAdapter)
  ↓
StripeService | MomoService (Adaptee)
```

BACKEND – ASP.NET Core Web API

Tạo project

```
dotnet new webapi -n PaymentAdapterDemo
cd PaymentAdapterDemo
```

CẤU TRÚC THƯ MỤC

```
PaymentAdapterDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   └── OrderService.cs
├── Payments
│   ├── IPayment.cs                (Target)
│   ├── StripeAdapter.cs           (Adapter)
│   ├── MomoAdapter.cs             (Adapter)
│   └── External
│       ├── StripeService.cs        (Adaptee)
│       └── MomoService.cs          (Adaptee)
├── Models
│   ├── CheckoutRequest.cs
│   └── PaymentResult.cs
└── Program.cs
```

1. TARGET INTERFACE (HỆ THỐNG CHUẨN)

Payments/IPayment.cs

```
namespace PaymentAdapterDemo.Payments;

public interface IPayment
{
    Task<PaymentResult> Pay(decimal amount);
}
```

2. MODEL

Models/PaymentResult.cs

```
namespace PaymentAdapterDemo.Models;

public class PaymentResult
```

```
{
    public bool Success { get; set; }
    public string Message { get; set; }
}
```

❧ 3. ADAPTEE (HỆ THỐNG NGOÀI – KHÔNG SỬA ĐƯỢC)

Stripe (API khác hoàn toàn)

```
namespace PaymentAdapterDemo.Payments.External;

public class StripeService
{
    public string MakeCharge(double money)
    {
        return $"Stripe charged {money}";
    }
}
```

Momo

```
namespace PaymentAdapterDemo.Payments.External;

public class MomoService
{
    public bool SendPayment(decimal amount)
    {
        return true;
    }
}
```

❧ 4. ADAPTER (CHUYỂN ĐỔI INTERFACE)

StripeAdapter

```
using PaymentAdapterDemo.Models;
using PaymentAdapterDemo.Payments.External;

namespace PaymentAdapterDemo.Payments;

public class StripeAdapter : IPayment
{
    private readonly StripeService _stripe;
```

```

public StripeAdapter()
{
    _stripe = new StripeService();
}

public async Task<PaymentResult> Pay(decimal amount)
{
    await Task.Delay(300);

    var result = _stripe.MakeCharge((double)amount);

    return new PaymentResult
    {
        Success = true,
        Message = result
    };
}
}

```

MomoAdapter

```

using PaymentAdapterDemo.Models;
using PaymentAdapterDemo.Payments.External;

namespace PaymentAdapterDemo.Payments;

public class MomoAdapter : IPayment
{
    private readonly MomoService _momo;

    public MomoAdapter()
    {
        _momo = new MomoService();
    }

    public async Task<PaymentResult> Pay(decimal amount)
    {
        await Task.Delay(300);

        var success = _momo.SendPayment(amount);

        return new PaymentResult
        {
            Success = success,
            Message = success ? "Paid via Momo" :
"Payment failed"

```

```
        };  
    }  
}
```

❁ 5. ORDER SERVICE

```
using PaymentAdapterDemo.Models;  
using PaymentAdapterDemo.Payments;  
  
namespace PaymentAdapterDemo.Services;  
  
public class OrderService  
{  
    public async Task<PaymentResult> Checkout(string  
method, decimal amount)  
    {  
        IPayment payment = method.ToLower() switch  
        {  
            "stripe" => new StripeAdapter(),  
            "momo" => new MomoAdapter(),  
            _ => throw new Exception("Unsupported  
payment")  
        };  
  
        return await payment.Pay(amount);  
    }  
}
```

❁ 6. REQUEST MODEL

```
namespace PaymentAdapterDemo.Models;  
  
public class CheckoutRequest  
{  
    public string Method { get; set; }  
    public decimal Amount { get; set; }  
}
```

❁ 7. CONTROLLER

```
using Microsoft.AspNetCore.Mvc;  
using PaymentAdapterDemo.Models;  
using PaymentAdapterDemo.Services;
```

```
namespace PaymentAdapterDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController()
    {
        _service = new OrderService();
    }

    [HttpPost("checkout")]
    public async Task<IActionResult>
    Checkout(CheckoutRequest request)
    {
        var result = await _service.Checkout(
            request.Method,
            request.Amount
        );

        return Ok(result);
    }
}
```

8. PROGRAM.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();
app.Run();
```

CHẠY BACKEND

```
dotnet run
```

Swagger:

`https://localhost:xxxx/swagger`

TEST API

POST:

`/api/orders/checkout`

Body:

```
{
  "method": "stripe",
  "amount": 100
}
```

LƯỒNG XỬ LÝ

Khi chọn STRIPE

```
Controller → OrderService
→ StripeAdapter (Adapter)
→ StripeService (Adaptee)
→ return kết quả
```

Khi chọn MOMO

```
Controller → OrderService
→ MomoAdapter
→ MomoService
→ return kết quả
```

SO SÁNH TRƯỚC & SAU ADAPTER

Trước	Sau
Phụ thuộc API ngoài	Chuẩn hóa qua IPayment
Code bị rối	Code sạch
Khó mở rộng	Dễ thêm adapter mới
Tight coupling	Loose coupling

KIẾN THỨC RÚT RA

👉 Adapter Pattern dùng khi:

- Interface KHÔNG tương thích
- Không sửa được code cũ (legacy / third-party)
- Muốn tái sử dụng code

BÀI TẬP MỞ RỘNG (CHO SINH VIÊN)

1. Thêm adapter:
 - ZaloPayAdapter
 - PayOSAdapter
2. Refactor:
 - Dùng DI thay vì new
 - Kết hợp với Factory Pattern
3. Nâng cấp:
 - Log request/response
 - Retry khi payment fail